

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе № 7
по дисциплине «Вычислительная математика»
Тема: Решение СЛАУ методом Гаусса

Студентка гр. 4341

Кочененко А.А.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Задание

В ходе выполнения работы необходимо реализовать метод Гаусса для решения системы линейных алгебраических уравнений с n неизвестными. Выполнение работы состоит из следующих этапов:

1. Разработать программу на языке Python, реализующую метод Гаусса с частичным выбором главного элемента (по столбцу). Программа должна предусматривать:

- генерацию матрицы коэффициентов и вектора свободных членов (случайная матрица, матрица Гильберта, диагонально-доминантная матрица);
- решение системы методом Гаусса;
- вычисление относительной невязки решения в L2-норме;
- вычисление числа обусловленности матрицы.

2. Провести тестирование разработанной программы на:

- маленьких наглядных матрицах (размером 3×3);
- больших случайных матрицах (размером 1000×1000 , 5000×5000 , 10000×10000);
- плохо обусловленных матрицах (матрица Гильберта).

3. Для проверки правильности реализации сравнить полученные решения с результатами встроенной библиотеки LAPACK (функция `scipy.linalg.solve`).

4. Исследовать обусловленность задачи: для каждого типа матрицы вычислить число обусловленности $\text{cond}(A)$. Проанализировать влияние числа обусловленности на точность полученного решения.

Теоретические положения

Метод Гаусса применяется для решения систем линейных алгебраических уравнений вида: $AX = b$, где A – матрица коэффициентов, X – вектор неизвестных, b – вектор свободных членов.

Метод включает в себя два этапа:

1. Прямой ход (преобразование к треугольному виду расширенной матрицы)

Прямой ход состоит из следующих шагов:

а) Перестановка строк для выбора главного элемента

При переходе к очередному столбцу матрицы необходимо, чтобы на диагонали (ведущий элемент строки) был ненулевой и наибольший по модулю элемент. В данной работе реализован метод Гаусса с частичным выбором главного элемента: на k -м шаге поиск максимального по модулю элемента ведётся в k -м столбце среди строк от k до n . Найденная строка переставляется с k -й строкой. Это позволяет уменьшить накопление ошибки округления.

б) Нормировка ведущей строки

Ведущая строка делится на главный элемент, чтобы этот элемент стал равен 1. Это упрощает последующие вычисления и делает обратный ход более наглядным.

в) Обнуление элементов под главным элементом

С помощью вычитания ведущей строки, умноженной на соответствующий коэффициент, обнуляются все элементы ниже главного элемента в текущем столбце. В результате получается ступенчатая (верхнетреугольная) матрица.

2. Обратный ход (подстановка)

Обратный ход выполняется следующим образом:

а) Вычисление значений неизвестных начинается с последнего уравнения и идёт снизу вверх.

б) Найденные значения последовательно подставляются в вышестоящие уравнения.

Результатом обратного хода является вектор X – решение системы.

Нормы векторов

Для оценки точности решения используется понятие нормы – способа измерить "размер" вектора. В данной работе применяется L2-норма (евклидова):

$$\|x\|_2 = \sqrt{(x_1^2 + x_2^2 + \dots + x_n^2)}$$

Эта норма выбрана, потому что она является стандартной для вычисления числа обусловленности матрицы и соответствует встроенной функции `np.linalg.cond(A)`.

Число обусловленности матрицы

Число обусловленности матрицы A определяется как:

$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$$

Оно показывает, насколько решение системы чувствительно к погрешностям исходных данных:

- если $\text{cond}(A)$ близко к 1 – матрица хорошо обусловлена, решение устойчиво;
- если $\text{cond}(A)$ велико (10^6 и более) – матрица плохо обусловлена, даже малые ошибки во входных данных могут привести к большим ошибкам в решении.

В работе число обусловленности вычисляется с помощью функции `np.linalg.cond(A)` в L2-норме.

Относительная невязка

После того как найдено приближенное решение $\hat{x}$, необходимо оценить, насколько хорошо оно удовлетворяет исходной системе $Ax = b$.

Для этого вычисляется вектор невязки: $r = A\hat{x} - b$

Если решение точное, то $r = 0$. Чем ближе r к нулю, тем точнее решение.

Чтобы получить одну количественную характеристику, используется норма вектора. А чтобы оценка не зависела от масштаба задачи, применяется относительная невязка:

$$\delta = \|A\hat{x} - b\|_2 / \|b\|_2$$

Чем меньше значение δ , тем точнее найденное решение. В идеальном случае (точное решение) $\delta = 0$.

Выполнение работы

В ходе выполнения работы была разработана программа на языке Python, реализующая метод Гаусса с частичным выбором главного элемента. Программа поддерживает два режима работы: исследование (*research*) и решение конкретной системы (*solve*). Ниже приведено описание основных функций.

1. *parse_args()* – функция создаёт и обрабатывает аргументы командной строки (CLI). Позволяет пользователю задавать размер матрицы n , точность вычислений *precision*, тип матрицы *type* (случайная, Гильберта, диагонально-доминантная) и интервал генерации случайных чисел *interval*. Поддерживается два режима: *research* для автоматического тестирования трёх типов матриц и *solve* для решения конкретной системы.

2. *validate_input(args)* – функция проверяет корректность входных данных. Контролируется, чтобы размер матрицы был не менее 3, точность не была отрицательной, а интервал находился в пределах от 0 до 1 000 000.

3. *generate_extended_matrix(n, precision, interval)* – функция генерирует случайную расширенную матрицу $[A|b]$. Элементы матрицы коэффициентов и вектора свободных членов заполняются случайными числами от 0 до *interval* с округлением до *precision* знаков после запятой.

4. *generate_hilbert_matrix(n, precision, interval)* – функция генерирует расширенную матрицу Гильберта $[A|b]$. Элементы матрицы вычисляются по формуле $H_{ij} = 1/(i + j - 1)$. Матрица Гильберта является классическим примером плохо обусловленной матрицы. Вектор свободных членов генерируется случайным образом.

5. *generate_diagonal_dominant(n, precision, interval)* – функция генерирует диагонально-доминантную расширенную матрицу $[A|b]$. Сначала создаётся случайная матрица, затем для каждой строки диагональный элемент увеличивается на сумму всех остальных элементов строки. Это обеспечивает

выполнение условия диагонального преобладания: $|a_{ii}| > \sum |a_{ij}|$ для $j \neq i$. Такие матрицы являются хорошо обусловленными.

6. *straight_elimination*(*n*, *matrix*) – функция выполняет прямой ход метода Гаусса. На каждом шаге:

- производится поиск максимального по модулю элемента в текущем столбце среди строк от текущей до последней (частичный выбор главного элемента);
- при необходимости строки переставляются, чтобы максимальный элемент оказался на диагонали;
- ведущая строка нормируется (делится на главный элемент);
- элементы ниже главного обнуляются вычитанием ведущей строки, умноженной на соответствующий коэффициент.

В результате матрица приводится к верхнетреугольному виду.

7. *inverse_step*(*n*, *matrix*) – функция выполняет обратный ход метода Гаусса. Вычисление неизвестных начинается с последнего уравнения и идёт снизу вверх. Для каждого *i*-го уравнения из правой части вычитаются уже найденные значения неизвестных, умноженные на соответствующие коэффициенты. Результатом является вектор решений *X*.

8. *compute_residual_norm*(*A*, *x*, *b*) – функция вычисляет относительную невязку решения. Сначала вычисляется вектор невязки $r = A \cdot x - b$, затем его L2-норма $\|r\|_2 = \sqrt{\sum r_i^2}$, которая делится на L2-норму вектора свободных членов $\|b\|_2$. Относительная невязка позволяет оценить точность решения независимо от масштаба задачи.

9. *compute_conditional_number*(*A*, *x*, *b*) – функция вычисляет число обусловленности матрицы. Используется формула $\text{cond}(A) = \|A\|_2 \cdot \|A^{-1}\|_2$. Для нахождения обратной матрицы применяется `np.linalg.inv(A)`, для вычисления норм – `np.linalg.norm(A, ord=2)`. Число обусловленности показывает, насколько решение чувствительно к погрешностям входных данных.

10. *gauss*(*matrix*, *n*, *precision*) – основная функция, реализующая метод Гаусса. Выполняет прямой ход (*straight_elimination*) и обратный ход

(inverse_step). Для проверки правильности реализации полученное решение сравнивается с эталонным решением от библиотеки LAPACK (функция `scipy.linalg.solve`). Функция возвращает словарь, содержащий найденное решение, эталонное решение, относительную невязку и число обусловленности.

11. *exploration(n, precision, interval)* – функция проводит исследование обусловленности. Последовательно генерирует матрицы трёх типов (случайная, Гильберта, диагонально-доминантная), решает их методом Гаусса и выводит таблицу, содержащую для каждого типа матрицы относительную невязку и число обусловленности. Это позволяет наглядно сравнить, как структура матрицы влияет на точность решения.

Таблица 1 – Результаты тестирования метода Гаусса (n=10)

Структура матрицы	Результат метода	Результат SciPy
Случайная	x1 = -3.208847 x2 = 0.606234 x3 = -4.194372 x4 = 1.828419 x5 = 0.381524 x6 = 5.194898 x7 = -0.082523 x8 = 4.519550 x9 = 6.149711 x10 = -6.658946	x1 = -3.208847 x2 = 0.606234 x3 = -4.194372 x4 = 1.828419 x5 = 0.381524 x6 = 5.194898 x7 = -0.082523 x8 = 4.519550 x9 = 6.149711 x10 = -6.658946

Гильберта	$x_1 = -252037.717711$ $x_2 = 7117780.745572$ $x_3 = -47055650.529706$ $x_4 = 111088122.659096$ $x_5 = -80706713.796339$ $x_6 = -27129221.221833$ $x_7 = 15123555.909674$ $x_8 = 20663740.013596$ $x_9 = 37543053.989109$ $x_{10} = -36456742.200343$	$x_1 = -252037.717711$ $x_2 = 7117780.745581$ $x_3 = -47055650.529770$ $x_4 = 111088122.659393$ $x_5 = -80706713.797298$ $x_6 = -27129221.219791$ $x_7 = 15123555.906813$ $x_8 = 20663740.016248$ $x_9 = 37543053.987599$ $x_{10} = -36456742.199949$
Диагонально-доминантная	$x_1 = 0.086321$ $x_2 = -0.023081$ $x_3 = 0.095587$ $x_4 = -0.035682$ $x_5 = 0.053943$ $x_6 = 0.098641$ $x_7 = -0.001441$ $x_8 = 0.026173$ $x_9 = 0.065425$ $x_{10} = 0.093760$	$x_1 = 0.086321$ $x_2 = -0.023081$ $x_3 = 0.095587$ $x_4 = -0.035682$ $x_5 = 0.053943$ $x_6 = 0.098641$ $x_7 = -0.001441$ $x_8 = 0.026173$ $x_9 = 0.065425$ $x_{10} = 0.093760$

Исследование обусловленности

Число обусловленности матрицы $\text{cond}(A)$ показывает, насколько решение системы линейных уравнений чувствительно к погрешностям исходных данных. Чем больше число обусловленности, тем сильнее малые ошибки в матрице A или векторе b могут повлиять на решение. Если $\text{cond}(A)$ близко к 1, матрица считается хорошо обусловленной, решение устойчиво. Если $\text{cond}(A)$ велико (например, 10^6 и более), матрица плохо обусловлена, и даже незначительные погрешности могут привести к большим ошибкам в ответе.

Важно отметить, что число обусловленности определяется структурой матрицы, а не её размером. Две матрицы одного размера могут иметь кардинально разные числа обусловленности.

Таблица 2 – Влияние структуры матрицы на обусловленность и точность решения

Размер n	Тип матрицы	Относительная невязка	Число обусловленности
10	Случайная	3.88e-16	56.89
10	Гильберта	3.18e-11	1.55e+07
10	Диагонально-доминантная	1.02e-16	2.80
100	Случайная	6.72e-15	2027.37
100	Гильберта	6.87e-12	1.40e+07
100	Диагонально-доминантная	4.70e-16	2.41

1000	Случайная	2.72e-13	103275.40
1000	Гильберта	5.56e-12	2.32e+07
1000	Диагонально-доминантная	1.34e-15	2.12

Анализ результатов

Из таблицы видно, что число обусловленности определяется как структурой матрицы, так и (для некоторых типов) её размером.

Диагонально-доминантные матрицы (хорошо обусловленные) сохраняют $\text{cond}(A) \sim 2-3$ при увеличении n с 10 до 1000. Относительная невязка при этом остаётся на уровне $10^{-15} — 10^{-16}$, что соответствует машинной точности. Это объясняется тем, что диагональное преобладание обеспечивает устойчивость метода.

Случайные матрицы показывают рост числа обусловленности с увеличением размера: при $n=10$ $\text{cond} \approx 57$, при $n=100 \approx 2000$, при $n=1000 \approx 100000$. Невязка при этом также возрастает: от 10^{-16} до 10^{-13} , но остаётся приемлемой для практических задач.

Матрицы Гильберта (плохо обусловленные) уже при $n=10$ имеют $\text{cond}(A) \sim 1.55 \cdot 10^7$. При увеличении размера до 1000 число обусловленности остаётся в том же диапазоне ($1.4 \cdot 10^7 — 2.3 \cdot 10^7$). Невязка для них составляет $10^{-11} — 10^{-12}$, что на 4-5 порядков выше, чем для хорошо обусловленных матриц того же размера. Это связано с тем, что матрица Гильберта близка к вырожденной, и даже малые ошибки округления сильно влияют на решение.

Выводы

В ходе выполнения лабораторной работы была разработана программа на языке Python, реализующая метод Гаусса с частичным выбором главного элемента. Программа поддерживает генерацию матриц трёх типов (случайные, Гильберта, диагонально-доминантные) и позволяет решать СЛАУ с заданной точностью.

Корректность реализации подтверждена сравнением с эталонной библиотекой LAPACK (`scipy.linalg.solve`): во всех тестах результаты полностью совпали.

Проведено исследование обусловленности для матриц размером $n = 10, 100, 1000$. Установлено, что:

- Диагонально-доминантные матрицы хорошо обусловлены ($\text{cond} \sim 2-3$), невязка минимальна ($\sim 10^{-16}$);
- Случайные матрицы имеют умеренную обусловленность (cond от 57 до 10^5), невязка возрастает до $\sim 10^{-13}$;
- Матрицы Гильберта плохо обусловлены ($\text{cond} \sim 10^7$), невязка составляет $\sim 10^{-11} \text{ — } 10^{-12}$.

Таким образом, подтверждена прямая зависимость между числом обусловленности матрицы и точностью решения: чем хуже обусловленность, тем больше относительная невязка. Разработанная программа может использоваться для решения СЛАУ и анализа обусловленности в учебных целях.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lb7.py

```
import random
import scipy
import numpy as np
import argparse

def parse_args():
    parser = argparse.ArgumentParser(
        description="Решение СЛАУ методом Гаусса и анализ обусловленности
матриц",
        formatter_class=argparse.RawTextHelpFormatter,
        epilog="Пример использования \n"
        "    Режим исследования:\n"
        "        python main.py --n <размер> --precision <точность>\n"
        "        python main.py --n <размер> --precision <точность> --
interval <интервал>\n"
        "    Режим решения:\n"
        "        python main.py --n <размер> --precision <точность>\n"
        "        python main.py --n <размер> --precision <точность> --
type <тип> \n"
        "        python main.py --n <размер> --precision <точность> --
type <тип> --interval <интервал>\n"
    )

    subparsers = parser.add_subparsers(dest='command', required=True)

    research_parser = subparsers.add_parser(
        'research',
        help="Сравнение типов матриц и их обусловленности"
    )

    research_parser.add_argument(
        "--n",
        type=int,
        required=True,
        help="Размер матрицы (целое положительное число)"
    )

    research_parser.add_argument(
        "--precision",
        type=int,
        required=True,
        help="Точность вычислений (количество знаков после запятой)"
    )

    research_parser.add_argument(
        "--interval",
        type=int,
        required=False,
        default=100,
        help="Максимальное число для случайной генерации (по умолчанию
100)"
    )

    gauss_parser = subparsers.add_parser(
        'solve',
        help = "Решить конкретную систему линейных уравнений"
    )
```

```

gauss_parser.add_argument(
    "--n",
    type=int,
    required=True,
    help="Размер матрицы (целое положительное число)"
)

gauss_parser.add_argument(
    "--precision",
    type=int,
    required=True,
    help="Точность вычислений (количество знаков после запятой)"
)

gauss_parser.add_argument(
    "--type",
    type=str,
    required=False,
    choices=["random", "hilbert", "diagonal-dominant"],
    default="random",
    help="Тип матрицы:\n"
        " random - случайная матрица\n"
        " hilbert - матрица Гильберта\n"
        " diagonal-dominant - диагонально-доминантная матрица\n"
        " по умолчанию (random)"
)

gauss_parser.add_argument(
    "--interval",
    type=int,
    required=False,
    default=100,
    help="Максимальное число для случайной генерации (по умолчанию
100)"
)

return parser.parse_args()

def validate_input(args):
    if args.n < 3: raise ValueError("Размер матрицы должен быть больше 3")
    if args.precision < 0: raise ValueError("Точность не может быть
отрицательной")
    if args.interval < 0 or args.interval > 1000000: raise
ValueError("Интервал должен быть от 0 до 1000000")

    def generate_extended_matrix(n, precision, interval):
        matrix = [[round(random.uniform(0, interval), precision) for _ in
range(n)] for _ in range(n)]
        vectorB = [round(random.uniform(0, interval), precision) for _ in
range(n)]
        return [matrix[i] + [vectorB[i]] for i in range(n)]

    def generate_hilbert_matrix(n, precision, interval):
        matrix = [[round(1 / (i + j - 1), precision) for j in range(1, n + 1)]
for i in range(1, n + 1)]
        b = [round(random.uniform(0, interval), precision) for _ in range(n)]
        return [matrix[i] + [b[i]] for i in range(n)]

    def generate_diagonal_dominant(n, precision, interval):
        matrix = [[round(random.uniform(0, interval), precision) for _ in
range(n)] for _ in range(n)]
        for i in range(n):
            row_sum = sum([matrix[i][j] for j in range(n) if i != j])
            matrix[i][i] += round(row_sum, precision)
        b = [round(random.uniform(0, interval), precision) for _ in range(n)]
        return [matrix[i] + [b[i]] for i in range(n)]

```

```

def straight_elimination(n, matrix):
    row_count = n
    column_count = n + 1
    for pivot_column in range(n):
        max_row = pivot_column
        for row in range(pivot_column + 1, row_count):
            if abs(matrix[row][pivot_column]) > abs(matrix[max_row]
[pivot_column]):
                max_row = row

        if abs(matrix[max_row][pivot_column]) == 0:
            raise ValueError("Система вырождена - ведущий элемент
нулевой")

        matrix[pivot_column], matrix[max_row] = matrix[max_row],
matrix[pivot_column]

        pivot = matrix[pivot_column][pivot_column]
        for i in range(pivot_column, column_count):
            matrix[pivot_column][i] /= pivot

        for row in range(pivot_column + 1, row_count):
            factor = matrix[row][pivot_column]
            for col in range(pivot_column, column_count):
                matrix[row][col] -= factor * matrix[pivot_column][col]

def inverse_step(n, matrix):
    x = [0.0] * n
    for i in range(n - 1, -1, -1):
        if abs(matrix[i][i]) == 0: continue
        x[i] = matrix[i][n]
        for c in range(i + 1, n):
            x[i] -= matrix[i][c] * x[c]
    return x

def gauss(matrix, n, precision):
    A = [row[:-1] for row in matrix]
    b = [row[-1] for row in matrix]
    straight_elimination(n, matrix)
    my_solution = inverse_step(n, matrix)
    solution_scipy = scipy.linalg.solve(A, b)
    return {
        "мое решение": [round(x, precision) for x in my_solution],
        "решение_scipy": [round(x, precision) for x in solution_scipy],
        "относительная невязка": compute_residual_norm(A, my_solution, b),
        "число обусловленности мое": round(compute_conditional_digit(A,
my_solution, b), precision),
        "число обусловленности_scipy": round(np.linalg.cond(A), precision)
    }

def compute_residual_norm(A, x, b):
    n = len(A)
    residual = [0.0] * n
    for i in range(n):
        Ax = sum(A[i][j] * x[j] for j in range(n))
        residual[i] = Ax - b[i]
    residual_norm = sum(r ** 2 for r in residual) ** 0.5
    b_norm = sum(b_i ** 2 for b_i in b) ** 0.5
    return residual_norm / b_norm

def compute_conditional_digit(A, x, b):
    A_inv = np.linalg.inv(A)
    norm_A = np.linalg.norm(A, ord=2)
    norm_A_inv = np.linalg.norm(A_inv, ord=2)
    return norm_A * norm_A_inv

```

```

def exploration(n, precision, interval):
    names = ["Случайная", "Гильберта", "Диагонально-доминантная"]
    generators = [
        generate_extended_matrix,
        generate_hilbert_matrix,
        generate_diagonal_dominant
    ]

    results = []
    for name, generator in zip(names, generators):
        matrix = generator(n, precision, interval)
        gauss_result = gauss(matrix, n, precision)
        results.append({
            "тип_матрицы": name,
            "относительная_невязка":
gauss_result["относительная_невязка"],
            "число_обусловленности_мое":
gauss_result["число_обусловленности_мое"],
            "число_обусловленности_scipy":
gauss_result["число_обусловленности_scipy"]
        })
        print(f"{'Тип матрицы':<25}{'Относительная невязка':<30}{'Число
обусловленности (моё)':<25}{'Число обусловленности (SciPy)':<25}")
        print("-" * 128)
        for result in results:
            print(f"{'тип_матрицы':<25}
{'относительная_невязка':<30}{'число_обусловленности_мое':<25}
{'число_обусловленности_scipy':<25}")

def main():
    args = parse_args()
    validate_input(args)
    if args.command == "research":
        exploration(args.n, args.precision, args.interval)
    elif args.command == "solve":
        matrix_generator = {
            "random": generate_extended_matrix,
            "hilbert": generate_hilbert_matrix,
            "diagonal-dominant": generate_diagonal_dominant
        }[args.type]
        system = matrix_generator(args.n, args.precision, args.interval)
        solution = gauss(system, args.n, args.precision)

        мое_решение = solution['мое_решение']
        решение_scipy = solution['решение_scipy']
        print(f"{'Индекс':<6}{'Моё решение':<20}{'Решение SciPy':<20}")
        print("-" * 46)
        for i, (мое, библиотечное) in enumerate(zip(мое_решение,
решение_scipy), start=1):
            print(f"x{i:<5}{str(мое):<25}{str(библиотечное):<30}")
            print("-" * 46)
            print(f"{'Относительная невязка':<30}
{solution['относительная_невязка']:.5e}")
            print(f"{'Число обусловленности (моё)':<30}
{solution['число_обусловленности_мое']:.5e}")
            print(f"{'Число обусловленности (SciPy)':<30}
{solution['число_обусловленности_scipy']:.5e}")

if __name__ == "__main__":
    main()

```